Занятие 4. Глубокое обучение: от нейрона к нейросети

Нейросети, обратное распространение, регуляризация, свёрточные сети и автоэнкодеры — с прицелом на задачи безопасности

нейрон MLP backprop CNN автоэнкодер

Листайте слайды стрелками ← →

Цели занятия

- Понять устройство нейрона и многослойной сети: прямое распространение.
- Разобрать обучение: функция потерь, градиент, обратное распространение, SGD.
- Знать практические приёмы: learning rate, регуляризация, BatchNorm, инициализация.
- Обзорно познакомиться с CNN, RNN/трансформерами и автоэнкодерами.
- Понять, где глубокое обучение применимо в задачах безопасности и каковы его пределы.

Программа занятия

Теория

- Нейрон, активации, архитектура MLP
- Потери, градиентный спуск, backprop
- Регуляризация и устойчивое обучение
- CNN, RNN, трансформеры — обзорно
- Автоэнкодеры и adversarial-риски

Практика

- MLPClassifier на данных digits
- Цикл обучения на PyTorch
- Автоэнкодер для поиска аномалий
- Метрики и валидация (повторение занятия 2)

Мостик от занятий 2–3

Занятие 2: классический ML

- Линейные модели, kNN, деревья, ансамбли.
- Признаки готовит человек (feature engineering).
- Предел: качество упирается в качество признаков.

Занятие 3: без учителя

- PCA и автоэнкодер — сжатие представлений.
- Автоэнкодер — мост к глубокому обучению.

Вопрос занятия 4: **можно ли научиться самим признакам, а не получать их от человека?**

Признаки вручную vs признаки из данных

- Классический ML: эксперт придумывает признаки → модель учит веса.
- Глубокое обучение: сеть сама строет иерархию представлений из сырых данных.
- Цена: нужно много данных, вычислений и аккуратности.

$$x \longrightarrow \text{слой}_1 \longrightarrow \text{слой}_2 \longrightarrow \ldots \longrightarrow \hat{y}$$

Каждый слой извлекает всё более абстрактные признаки: края → формы → объекты.

Где глубокое обучение выигрывает

Изображения

Классификация, детекция, сегментация: свёрточные сети (CNN).

Текст и речь

Эмбеддинги, трансформеры: NLP, перевод, классификация обращений.

Последовательности

Логи, сетевой трафик, метрики: аномалии и прогнозирование.

Нейрон: базовый строительный блок

Формула нейрона

$$z = \langle w, x \rangle + b$$

$$a = \varphi(z)$$

- w — веса, b — смещение;
- φ — функция активации (нелинейность).

Интуиция

- Нейрон = знакомая линейная модель из занятия 2...
- ...плюс нелинейная активация на выходе.
- Без активации любая сеть оставалась бы линейной моделью.

Функции активации

Sigmoid / tanh

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Сжимают в (0, 1) / (−1, 1); страдают от затухающих градиентов.

ReLU

$$\mathrm{ReLU}(z) = \max(0, z)$$

Стандарт скрытых слоёв: дешёвая и без затухания градиента.

Softmax (выход)

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Превращает логиты (сырые выходы нейронов) в вероятности классов.

Линейная (выход)

$$\varphi(z) = z$$

Для задач регрессии на выходе сети.

Зачем нужна нелинейность: пример XOR

- Один линейный слой разделяет данные только прямой (гиперплоскостью).
- XOR линейно неразделим — вспомните занятие 2 и семинары.
- Композиция нелинейностей строит сколь угодно сложные границы.

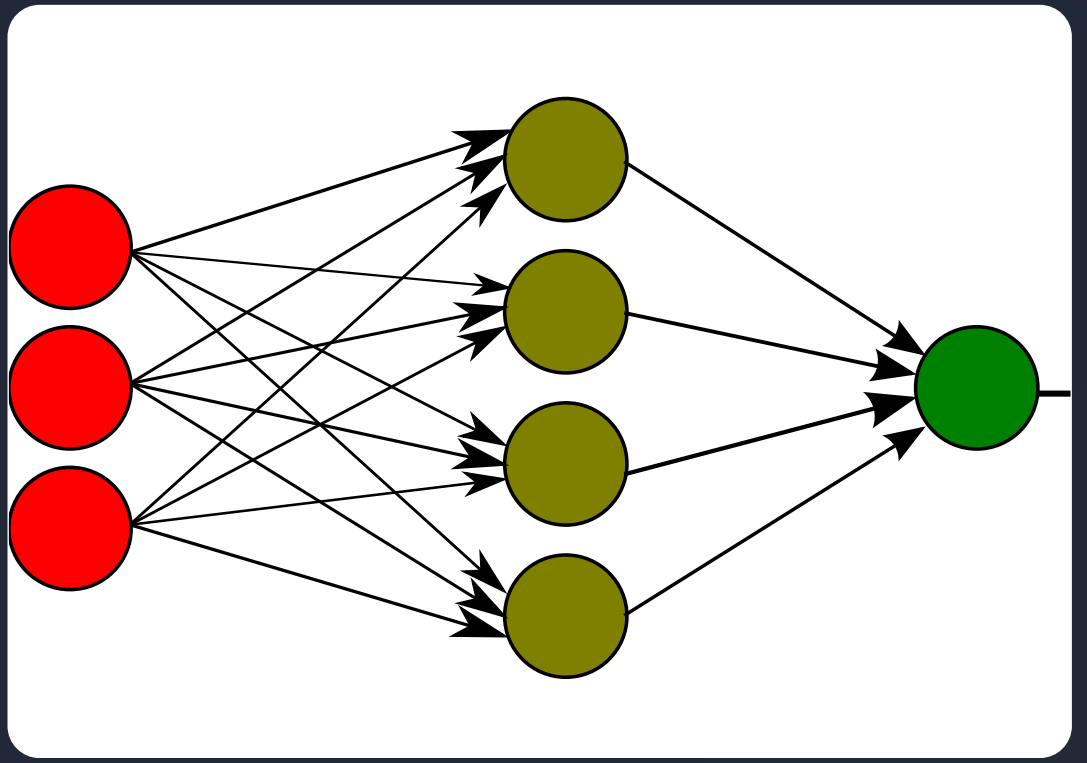
$$\hat{y} = \varphi_2\big(W_2\,\varphi_1(W_1 x + b_1) + b_2\big)$$

Два слоя с активацией уже решают XOR.

Архитектура: слои и глубина

Структура сети

- Входной слой: сырые признаки (пиксели, эмбеддинги).
- Скрытые слои: извлечение представлений.
- Выходной слой: предсказание (класс, число).
- «Глубокая» = много скрытых слоёв.



Многослойный перцептрон (MLP): слои и связи между нейронами

Прямое распространение (forward pass)

$$h^{(l)} = \varphi\big(W^{(l)} h^{(l-1)} + b^{(l)}\big), \qquad h^{(0)} = x$$

- Каждый слой: матричное умножение + активация.
- Вычисления идут последовательно от входа к выходу.
- Именно этот проход даёт предсказание модели.

Параметры сети

- Учатся только веса W и смещения b; архитектуры и активации задаёт человек.
- Слой $n \times m$ имеет $m \cdot n + m$ параметров.
- Миллионы и миллиарды параметров — норма для современных сетей.

Пример: слой 784→128 (digits) = 784·128 + 128 = 100 480 параметров.

Функция потерь: регрессия

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \quad \text{(MSE)}$$

- Потеря измеряет ошибку сети на всей выборке.
- Обучение = минимизация потерь по параметрам.
- Для регрессии — MSE/MAE, вспомните занятие 2.

Функция потерь: классификация

Бинарная (cross-entropy)

$$L = -\frac{1}{n}\sum_{i}\left[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)\right]$$

Мультикласс (softmax + CE)

$$L = -\frac{1}{n}\sum_{i} \ln \hat{y}_{i,\,y_i}$$

Штрафуем вероятность правильного класса.

Обучение = оптимизация

$$w \leftarrow w - \eta \nabla_w L$$

- ∇L — градиент: направление наискорейшего роста потерь.
- Шаг в противоположную сторону уменьшает потерю.
- η — learning rate: размер шага.

Интуиция градиента

- Частная производная $\frac{\partial L}{\partial w_j}$ — насколько потеря чувствительна к весу w_j.
- Аналогия: спуск с холма по самому крутому склону.
- Проблема: сеть — композиция сотен операций; как посчитать производные?

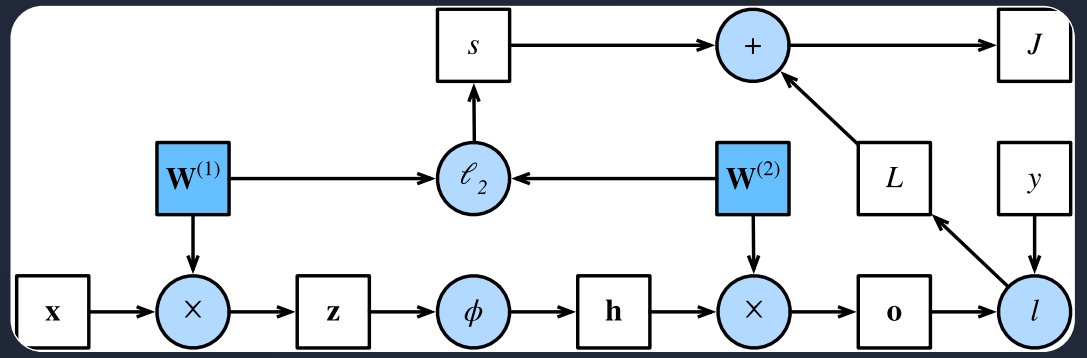
Ответ даёт правило дифференцирования сложной функции — **chain rule**.

Обратное распространение (backprop)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- Ошибка propagates от выхода ко входу по цепочке слоёв.
- За один обратный проход получаем градиенты *всех* параметров.
- Это лишь аккуратное применение chain rule, а не магия.

Вычислительный граф



Forward считает потерю, backward — градиенты по всем узлам графа

Что делает фреймворк

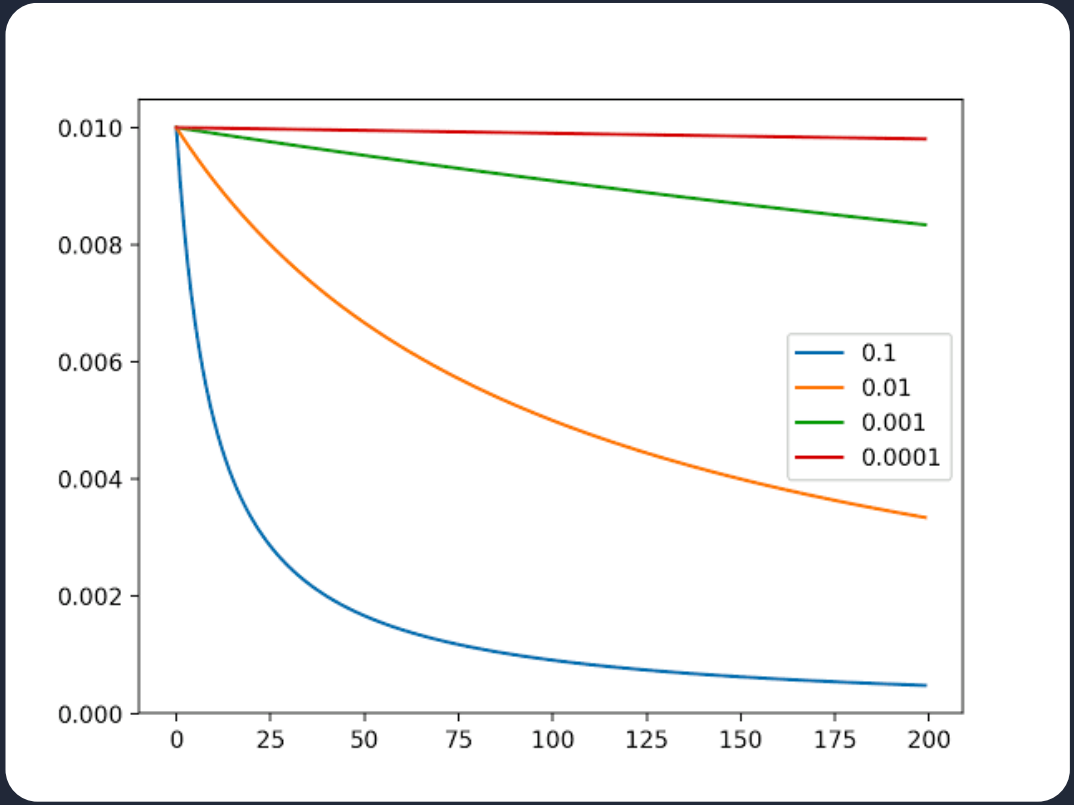
- Строит граф операций при прямом проходе.
- `loss.backward()` обходит граф в обратную сторону.
- Человек не пишет производные вручную.

SGD и мини-батчи

$$w \leftarrow w - \eta \nabla_w L_{\text{batch}}$$

- Полный градиент по всей выборке дорог: считаем по мини-батчу (32–256 объектов).
- Стохастичность шумит, но помогает escapar из плоских минимумов.
- Один проход по выборке = одна **эпоха**.

Learning rate: главный гиперпараметр



Слишком большой η — расходимость; слишком малый — вязкий спуск

Практика

- Начинают с 1e-3 (Adam) и подбирают по валидации.
- Schedule: уменьшение η по ходу обучения.
- Кривые потерь train/val — главный диагностический инструмент.

Оптимизаторы: momentum и Adam (обзорно)

Momentum

$$v \leftarrow \mu v + \nabla L, \qquad w \leftarrow w - \eta v$$

Накопленная «скорость» сглаживает колебания спуска.

Adam

- Адаптивный шаг для каждого параметра.
- Стандартный выбор по умолчанию в большинстве задач.

Эпохи, переобучение и валидация

- Сети с миллионами параметров легко запоминают выборку (переобучение, занятие 2).
- Следим за кривыми train/val: расхождение кривых — сигнал стоп.
- Early stopping: останавливаем обучение на лучшей эпохе по валидации.

Данные делим так же, как в занятии 2: train / val / test, fit только на train.

Регуляризация нейросетей

L2 (weight decay)

$$L_{reg} = L + \lambda \sum_j w_j^2$$

Штрафуем большие веса — та же идея, что в занятии 2.

Dropout

- На обучении случайно «выключаем» долю нейронов.
- Сеть не зависит от отдельных нейронов → ансамбль внутри.

BatchNorm: идея (обзорно)

- Нормализует активации внутри батча: среднее ≈ 0, разброс ≈ 1.
- Ускоряет и стабилизирует обучение, позволяет больше η.
- Побочный эффект — лёгкая регуляризация.
- Масштабирующие параметры γ, β учатся вместе с сетью.

Проблема затухающих градиентов

- В глубоких сетях с sigmoid градиенты к ранним слоям стремятся к нулю.
- Ранние слои перестают учиться.

Решения

- ReLU вместо sigmoid;
- аккуратная инициализация весов;
- BatchNorm;
- остаточные связи (ResNet — обзорно).

Инициализация весов (обзорно)

- Нули — нельзя: нейроны станут одинаковыми (симметрия).
- Xavier/Glorot — для sigmoid/tanh; He — для ReLU: $\sigma^2 = 2/n$.
- Правильный старт решает, сойдётся ли обучение вообще.

Во фреймворках инициализация по умолчанию уже разумная — но знать причину полезно.

Фреймворки: PyTorch и TensorFlow/Keras

PyTorch

- Pythonic-API, динамический граф.
- Стандарт в исследованиях и всё чаще в проде.

TensorFlow / Keras

- Зрелый продакшн: TF Serving, мобильный вывод.
- Keras — высокоуровневый API для быстрых экспериментов.

Оба дают **автоград**: автоматическое дифференцирование.

Автоград: производные без формул

- Фреймворк записывает операции прямого прохода в граф.
- Вызов `backward()` применяет chain rule ко всем узлам.
- Разработчик модели не выводит производные вручную.

Именно автоград сделал глубокое обучение инженерной дисциплиной, а не искусством дифференцирования.

Типовой пайплайн обучения

1. Данные и преобразования (нормализация!, занятие 2).
2. Модель: архитектура и размеры слоёв.
3. Функция потерь и оптимизатор.
4. Цикл: forward → loss → backward → step.
5. Валидация, метрики, early stopping.

Минимальный цикл обучения на PyTorch

```
import torch, torch.nn as nn

model = nn.Sequential(
    nn.Linear(64, 128), nn.ReLU(),
    nn.Linear(128, 64), nn.ReLU(),
    nn.Linear(64, 10),
)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    for Xb, yb in train_loader:
        opt.zero_grad()
        loss = loss_fn(model(Xb), yb)
        loss.backward()
        opt.step()
```

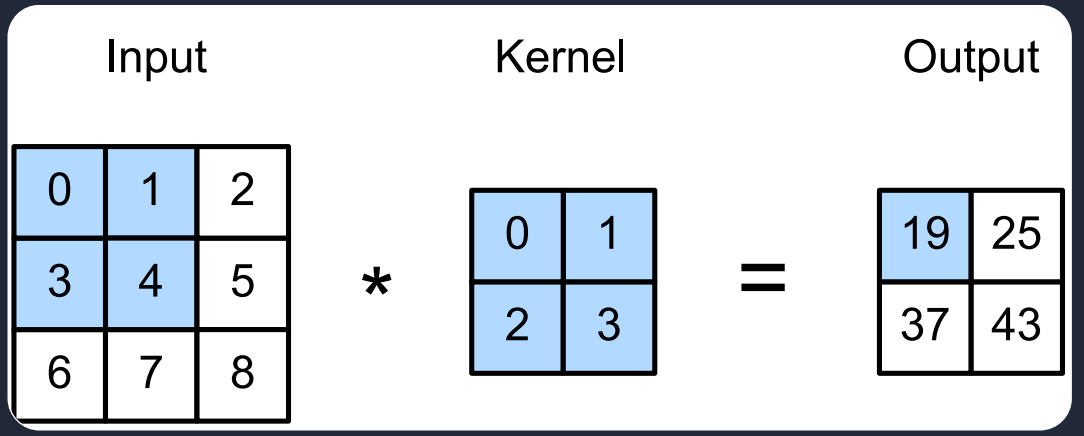
Валидация и метрики нейросетей

- Те же метрики, что в занятии 2: accuracy, precision/recall, ROC-AUC, F1.
- Дисбаланс классов: те же ловушки (accuracy бесполезна).
- Масштабирование признаков обязательно: сети чувствительны к масштабу.
- Сравниваем с базовыми моделями занятия 2 (kNN, RF) — дешёвый бенчмарк.

Свёрточные сети (CNN): идея

Что такое свёртка

- Маленький фильтр скользит по изображению.
- Каждый фильтр детектирует свой паттерн (край, текстура).
- Слои строят иерархию: края → формы → объекты.



Фильтр скользит по изображению и строит карту признаков

Свёртка формально

$$S(i,j) = \sum_m \sum_k I(i+m,\ j+k)\, K(m,k)$$

- K — ядро свёртки: небольшая матрица обучаемых весов.
- Веса общие для всей картинки (weight sharing) → параметров мало.
- Несколько фильтров → несколько карт признаков в одном слое.

Stride, padding, pooling (обзорно)

Stride

Шаг скольжения фильтра; stride > 1 уменьшает размер карты.

Padding

Добавление рамки нулей: сохраняем размер и информацию о краях.

Pooling

Уменьшение разрешения (max-pool): инвариантность к малым сдвигам.

Почему CNN лучше MLP на изображениях

- Локальность: паттерн виден в малой области, не во всей картинке.
- Инвариантность к сдвигу: объект узнаётся в любом месте.
- На порядок меньше параметров при том же качестве.

MLP на картинке 224×224×3 имел бы первый слой в миллионы весов без всякой структуры.

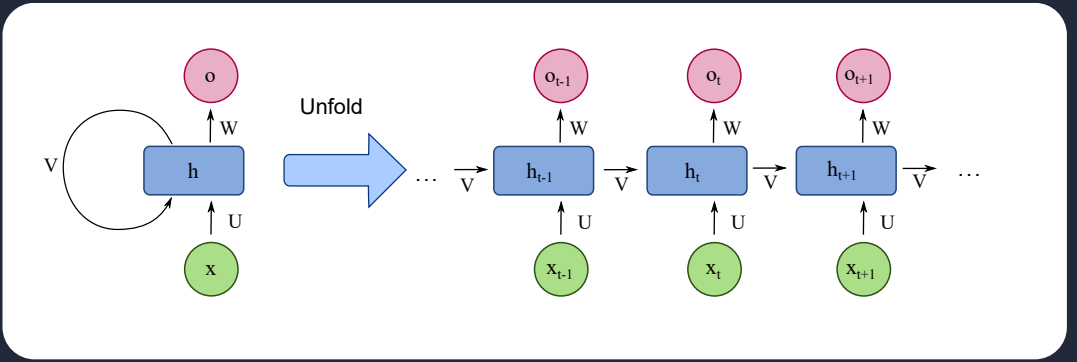
CNN в задачах безопасности (обзорно)

- Классификация вредоносных файлов (бинарники как изображения).
- Детекция аномального трафика по спектрограммам/представлениям пакетов.
- Распознавание лиц и объектов в видеоаналитике.
- Осторожно: качество упирается в размеченные данные и дрейф (занятие 3).

Рекуррентные сети (RNN) для последовательностей

Идея

- Сеть хранит скрытое состояние во времени.
- Обрабатывает логи, текст, трафик, метрики.
- Проблема: затухающий градиент на длинных последовательностях.



RNN: состояние передаётся от шага к шагу

LSTM/GRU и трансформеры (обзорно)

LSTM / GRU

Вентили против затухающего градиента: память на длинные зависимости.

Transformer

Механизм внимания (attention); параллельное обучение; стандарт NLP и всё чаще временных рядов.

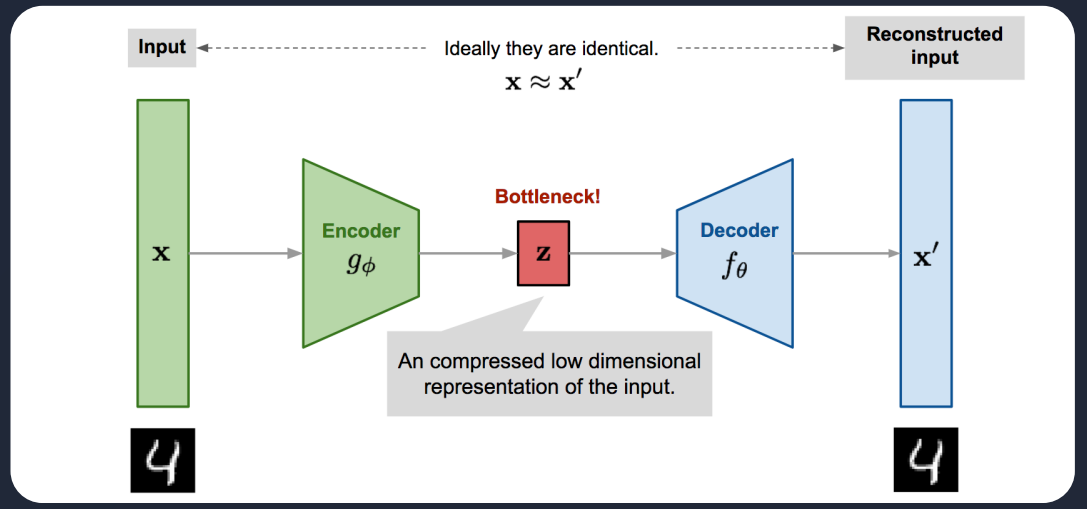
Детали трансформеров — тема отдельных занятий; здесь достаточно знать место в иерархии архитектур.

Эмбеддинги: векторные представления

- Объект (слово, запрос, событие) отображается в плотный вектор.
- Близкие по смыслу объекты близки в пространстве.
- Эмбеддинги можно подавать классическим моделям занятия 2 (kNN, RF).

Мост между глубоким обучением и классическим ML-пайплайном.

Автоэнкодер: мостик из занятия 3



Encoder сжимает вход в код, decoder восстанавливает вход

Формулы

$$\hat{x} = D(E(x))$$

$$L = \|x - \hat{x}\|^2$$

Узкое место (bottleneck) учит сжатое представление — как PCA, но нелинейное.

Автоэнкодер для поиска аномалий

- Учим на нормальных данных: они восстанавливаются хорошо.
- Аномалия даёт большую ошибку реконструкции $||x - \hat{x}||^2$.
- Порог ошибки — по квантили на валидации (приём из занятия 3).

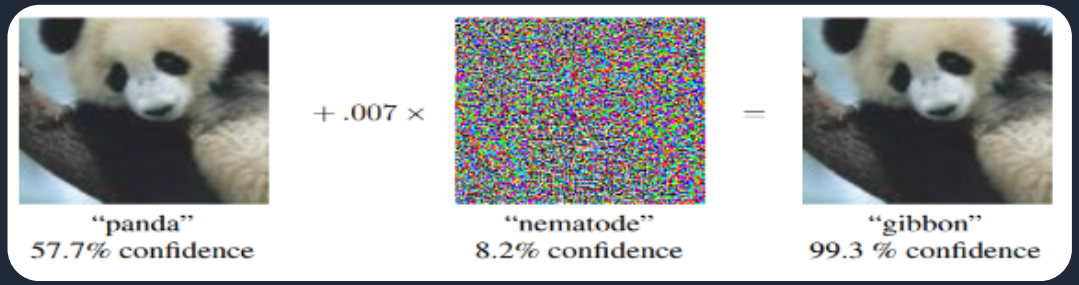
Связка: аномалии (занятие 3) + глубокое обучение (занятие 4) = рабочий детектор для сложных данных.

Adversarial-примеры (обзорно)

Атака FGSM

$$x' = x + \varepsilon \cdot \text{sign}(\nabla_x L(x, y))$$

Незаметное возмущение вдоль градиента меняет предсказание.



Классический пример: panda + шум → gibbon с уверенностью 99%

Устойчивость моделей: что делают на практике

- Adversarial-training: обучение на примерах с возмущениями.
- Контроль входных данных и мониторинг уверенности предсказаний.
- Ансамбли и детекторы выбросов на входе (мостик к занятию 3).
- Полной защиты нет: robustness — свойство, которое проектируют.

Пределы глубокого обучения

- Требует много размеченных данных и вычислений.
- Плохо интерпретируемо: «чёрный ящик».
- Чувствительно к дрейфу распределения (MLSecOps, занятие 3).
- Часто проигрывает градиентному бустингу на табличных данных.

Интерпретируемость (обзорно)

- Saliency-карты: какие пиксели повлияли на решение.
- Attention-веса: на что смотрел трансформер.
- SHAP/LIME: локальные объяснения предсказаний (идеи из занятия 2).
- В безопасности объяснимость — требование регуляторов и расследований.

Transfer learning и предобученные модели

- Берём сеть, предобученную на большом датасете (ImageNet, текстовые корпуса).
- Дообучаем (fine-tune) на своих данных — экономим данные и время.
- Стандартная практика: своя модель с нуля нужна редко.

Связь с MLSecOps

- DL-детекторы вредоносного ПО и аномального трафика.
- Автоэнкодеры для мониторинга метрик и логов (занятие 3).
- Adversarial-риски: атаки на сами ML-модели в проде.
- Жизненный цикл: дрейф, переобучение, деградация качества со временем.

Практика: план работы

Часть 1. MLP

sklearn MLPClassifier на digits; сравнение с kNN/RF из занятия 2.

Часть 2. PyTorch

Свой цикл обучения: Adam, dropout, early stopping; кривые потерь.

Часть 3. Автоэнкодер

Поиск аномалий по ошибке реконструкции; порог по квантили.

Практика: быстрый старт и чек-поинты

```
from sklearn.neural_network import MLPClassifier

clf = MLPClassifier(hidden_layer_sizes=(128, 64),
                    activation="relu", solver="adam",
                    max_iter=200, random_state=42)
clf.fit(X_train, y_train)
```

- Чек-поинт 1: accuracy/F1 против kNN и RF (занятие 2).
- Чек-поинт 2: кривые train/val — есть ли переобучение?
- Чек-поинт 3: precision@k автоэнкодера на синтетических выбросах (занятие 3).

Итоги занятия

- Нейрон = линейная модель + нелинейная активация; глубина = композиция.
- Обучение = минимизация потерь: градиент, backprop, SGD/Adam.
- Регуляризация и валидация работают так же, как в классическом ML.
- CNN — изображения, RNN/трансформеры — последовательности, автоэнкодер — мост к занятию 3.
- DL силён, но требует данных, вычислений и осторожности в задачах безопасности.

Следующий шаг: практический ноутбук — MLP на digits, цикл обучения на PyTorch и автоэнкодер для аномалий.